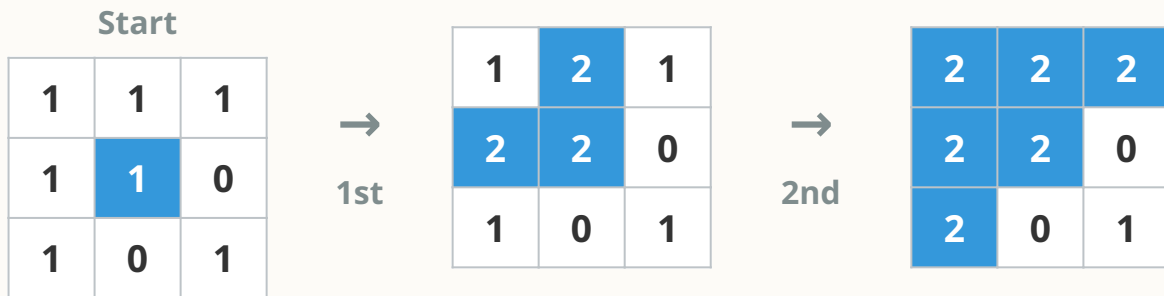


Graphs

Authored by Arpit Raj, LNMIIT Jaipur • July 21, 2026

LC733 Flood Fill (Palantir, OpenAI)



Input / Output

- **i/p** → image grid, sr, sc, coloring value
- **o/p** → 2D output grid

Algorithm 1 → DFS

1. store original color
2. if orig == new return
3. DFS
4. paint current cell
5. visit 4 neighbours

Note: DFS uses recursion stack

```

class Solution {
public:
    vector<pair<int,int>> dir{{-1,0},{0,-1},{0,1},{1,0}};
    int n;
    int m;

    void dfs(int r,int c,vector<vector<int>>& image,int orig,int col){

        if(image[r][c]==col) return;
        image[r][c]=col;
        for(auto it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nr<n && nc>=0 &&nc<m){
                if(image[nr][nc]==orig){
                    dfs(nr,nc,image,orig,col);
                }
            }
        }
    }

    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color) {
        n=image.size();
        m=image[0].size();

        int orig=image[sr][sc];
        dfs(sr,sc,image,orig,color);
        return image;
    }
};

```

Algorithm 2 → BFS

- keeps cells inside a queue
- paint one
- push its neighbours
- repeat until queue empty
- push only and only if cell color is == orig

Note: BFS uses queue

```

class Solution {
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int col) {

        int n=image.size();
        int m=image[0].size();

        int orig=image[sr][sc];
        if(orig==col){
            return image;
        }

        queue<pair<int,int>> q;
        vector<pair<int,int>> dir {{-1,0},{1,0},{0,-1},{0,1}};
        q.push({sr,sc});

        while(!q.empty()){
            int r=q.front().first;
            int c=q.front().second;
            image[r][c]=col;
            q.pop();

            for(auto it:dir){
                int nr=r+it.first;
                int nc=c+it.second;
                if(nr>=0 && nc>=0 && nr<n && nc<m){
                    if(image[nr][nc]==orig){
                        image[nr][nc]=col;
                        q.push({nr,nc});
                    }
                }
            }
        }

        return image;
    }
};

```

Complexity

both are $O(n \times m)$